Module 4 - Open Code

NASA Open Science 101

Open Science Code of Conduct

By participating in this workshop, I will:

- Conduct myself with integrity, respect, honesty, and credibility.
- Approach all events, activities, and discussions with the highest ethical standards of professionalism and personal conduct.
- Avoid personal attacks or other activities that cause damage to other participants or the hosts.
- Value and respect diversity of views and opinions.

Reporting Unacceptable Behavior:

- If you are the subject of unacceptable behavior or have witnessed any such behavior, please immediately notify a meeting host.
- Notification should be done by contacting a host via direct chat or emailing your concern to your workshop instructor.
- Anyone experiencing or witnessing behavior that constitutes an immediate or serious threat to public safety is advised to contact 911 or your local emergency number.

Agenda

I. Welcome Remarks
II. Lesson 1: Introduction to Open Code
III. Lesson 2: Using Open Code
IV. Lesson 3: Making Open Code
V. Lesson 4: Sharing Open Code
VI. Lesson 5: From Theory to Practice
VII. Closing Remarks

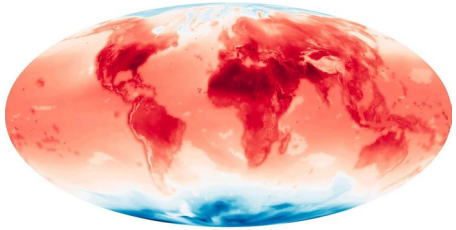
Module 4 - Lesson 1

Introduction to Open Code

Lesson 1: Learning Objectives

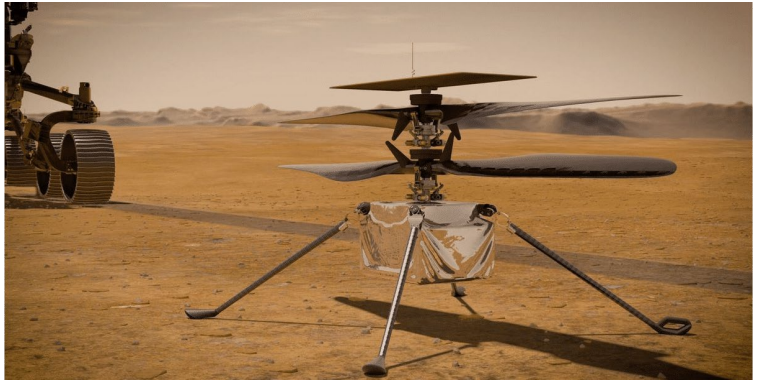
- We will define open-source software and distinguish it from closed-source software.
- We will list common benefits and challenges to the production of open code and describe how researchers can respond to some of the challenges while maximizing openness when appropriate.
- We will describe the function and purpose of a Software Management Plan, as its utility as a guidebook for everyone involved in a scientific project.

Success Stories

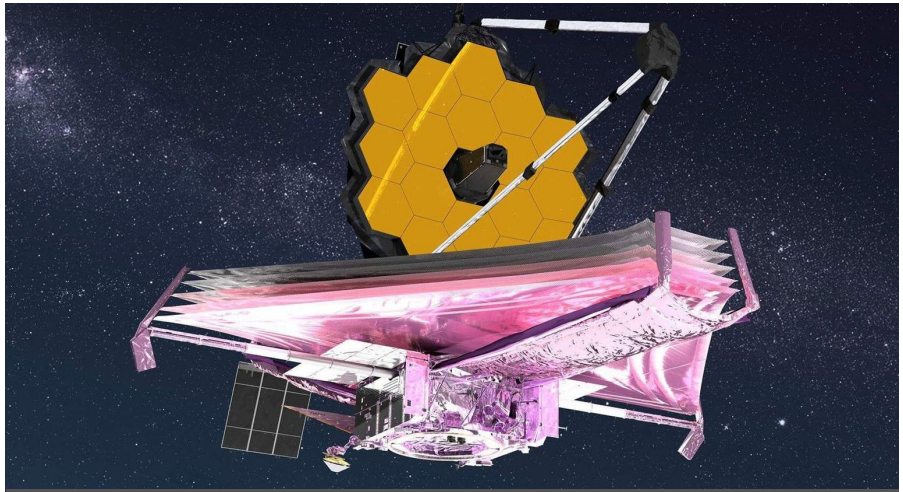


Open models to study the upper atmosphere have been repurposed to study the life cycles of weather systems and monsoons

Black hole imaging relied on open-source code



The Ingenuity helicopter runs on open source software



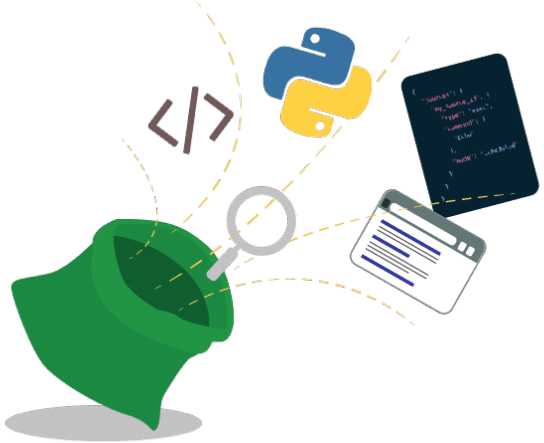
Some early James Webb Space Telescope (JWST) projects enabled open and collaborative access to their methods, increasing the speed at which they could turn around ground-breaking results.

What is Code vs Software

Code is a language that humans can type and understand.

Software is often a collection of programs, data, and other information that a computer system uses to perform specific tasks.

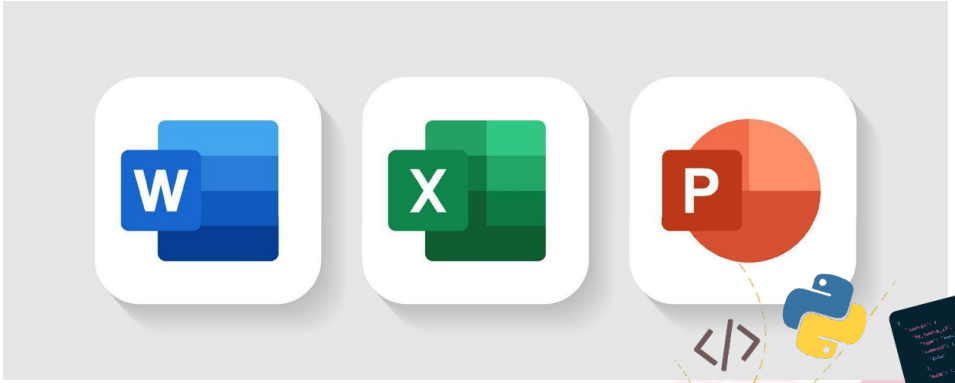
Open-source software is distributed with its source code without cost, making it available for others to use, modify, and distribute with its original rights and permissions.



Examples of Code vs Software

```python
def celsius_to_fahrenheit(celsius):
    return (celsius * 9/5) + 32

def fahrenheit_to_celsius(fahrenheit):
    return (fahrenheit - 32) * 5/9

def main():
    print("Temperature Conversion")
    print("1. Celsius to Fahrenheit")
    print("2. Fahrenheit to Celsius")
    choice = input("Enter your choice (1 or 2): ")

    if choice == '1':
        celsius = float(input("Enter temperature in Celsius: "))
        fahrenheit = celsius_to_fahrenheit(celsius)
        print(f"{celsius}°C is equal to {fahrenheit}°F")
    elif choice == '2':
        fahrenheit = float(input("Enter temperature in Fahrenheit: "))
        celsius = fahrenheit_to_celsius(fahrenheit)
        print(f"{fahrenheit}°F is equal to {celsius}°C")
    else:
        print("Invalid choice. Please enter 1 or 2.")

if __name__ == "__main__":
    main()
```

Computer software is a key component to allowing your devices to work as intended and help you complete your goals. There are several categories of software you may use at work depending on your needs.



Types of Software

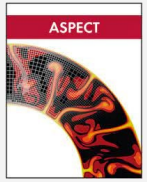
General Purpose Software





Modeling and Simulation Software

OpenFOAM



Operational Software Infrastructure Software



Analysis Software

photutils

An Astropy Package for Photometry

Libraries


Single Use Utility Software

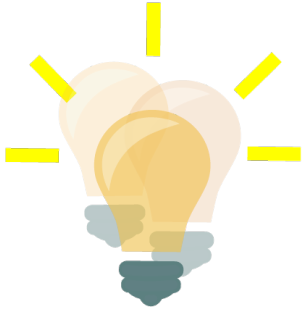
Credit: Stuart Heggie/JPL

Credit: NASA GSFC/CIL/Adriana Manrique Gutierrez

These are some examples of Software

- **Python (PyCharm)**
- **Anaconda**
- **MySQL**

Principles of Open Code



Transparency

Collaboration

Share early and often

Inclusive

Community

Benefits of Moving to Open Software

- **Accelerates science** by making it easier to use and build on software developed in previous work.
- **Minimizes the time and cost** of repeated development of similar software and the reproduction of scientific computations.
- **Increases the number of users and developers** by enabling easier access.
- **Increases visibility, sustainability, software quality** by increasing availability and collaborative oversight.

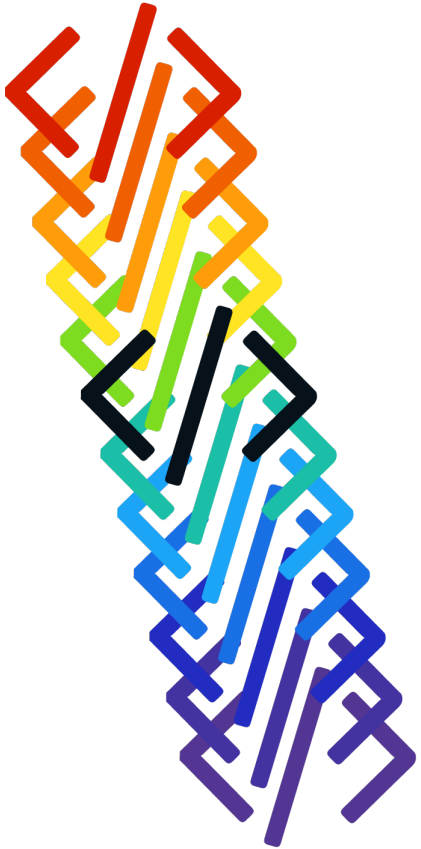
Activity 1.1

Scooping: What if someone re-uses my code to publish a result I was working on?	What if my code is misinterpreted or misused	What if my code is used but not cited?	What if my code is too sensitive to share?	What if I don't think my code will be useful to anyone else?

Open Code is a Spectrum

- There is a spectrum of openness when it comes to open software that ranges from open- source software to closed source software.
- Some projects may be open from inception and continuously share all code throughout development. Others may share some of the code at the time of publication. Other projects may only make code available once funding ends.
- Licensing (Lesson 3) is an important consideration in planning your open code.
- It is important to plan how the software will be made open at the beginning of the project with the data management plan
- Plans can and should be flexible

While researchers and institutions may not be able to share all their code, they can make efforts to shift on the openness spectrum from closed code to open-source code and software.



Sometimes it is not safe to share…

When Not to Share

- The code incorporates a country's military secrets or its dissemination violates national interests or security concerns.
- The code incorporates intellectual property or patented data and information.
- Institutional policies or organizational regulations do not permit the sharing of code.
- Think about what you are sharing and the implications of sharing it (for example - do you have permission from everyone involved?).

Software release & Version control

- Brief GitHub Overview to introduce software release and version control
  - GitHub is a web-based platform that uses Git for version control, enabling developers to collaborate on code projects.
  - GitHub has become a crucial tool in the software development industry.
  - It provides a repository hosting service that allows users to store, manage, and track changes to their code.
  - Key features:
    - Repositories
    - Branching and Merging
    - Pull Requests
    - Issue Tracking
    - Actions

Planning for Open Code

What?	Description of types, management, preservation, and release of software.
When?	The schedule for software archiving and sharing.
Where?	Location where software will be share and archived over the long term.
How?	Enable reuse of software through designing a DOI, license, contribution guidelines, etc.
Who?	Roles and responsibilities of the team members.

Software Management Plans (SMP) are a framework and may need to be adjusted during project execution.

Lesson 1: Knowledge Check (1 of 3)

Read the statement below and decide whether it's true or false:

Software is referred to as open source when it is publicly accessible; anyone can see, modify, and distribute the code as they see fit.

- True
- False

Lesson 1: Knowledge Check (1 of 3)

Read the statement below and decide whether it's true or false:

Software is referred to as open source when it is publicly accessible; anyone can see, modify, and distribute the code as they see fit.

- True
- False

Lesson 1: Knowledge Check (2 of 3)

Which of the following are valid reasons why scientists keep their source code closed? Select all that apply.

A. National security concerns
B. Institutional policies
C. Data privacy concerns
D. Attribution concerns
E. Quality concerns

Lesson 1: Knowledge Check (2 of 3)

Which of the following are valid reasons why scientists keep their source code closed? Select all that apply.

A. National security concerns
B. Institutional policies
C. Data privacy concerns
D. Attribution concerns
E. Quality concerns

Lesson 1: Knowledge Check (3 of 3)

What are the main sections in a software management plan?

A. Types of code and software
B. Schedule for sharing software
C. Where software will be shared and archived
D. What license it will be assigned
E. Roles and responsibilities of team members
F. All of the above

Lesson 1: Knowledge Check (3 of 3)

What are the main sections in a software management plan?

A. Types of code and software
B. Schedule for sharing software
C. Where software will be shared and archived
D. What license it will be assigned
E. Roles and responsibilities of team members
F. All of the above

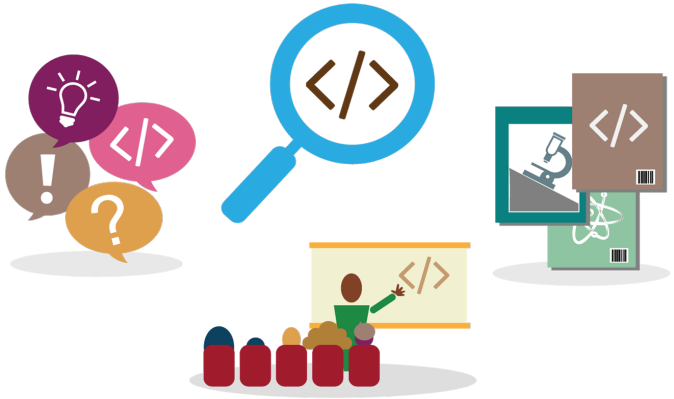
Module 4 - Lesson 2

Using Open Code

Lesson 2: Learning Objectives

- Describe the process of using open code and list some key elements of discovering code.
- Describe the four key considerations when assessing open software: functionality, interoperability, security, and licenses.
- List some common problems that arise when reusing Open Code and best practices to resolve them.
- Describe how, where, and under what circumstances one should acknowledge (cite) code.

Discovering Open Code and Software



Example	I'm a new graduate student starting to work on modeling turbulence in the Southern Ocean to better understand sea surface temperature (or ocean heat uptake) and climate change. Is there some software available to model how eddies in the ocean affect sea-surface temperature?
Exercise	General Search on the term "Software for ocean turbulence modeling"
Result	General Ocean Turbulence Model (GOTM)

This successful search is predicated on the developers of GOTM making their code open.

Open Software Discovery & Developers Following FAIR Principles

Discovering open software depends on developers making their software easy to find. The Findable, Accessible, Interoperable and Reusable (FAIR) Principles for research software suggest:

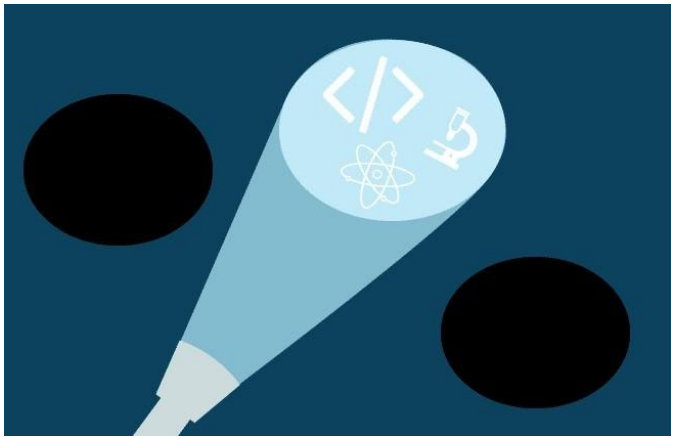
- Software and its associated metadata must be easy for humans and machines to find.
- Software must be described with rich, searchable, and indexable metadata.
- Software must be findable from all relevant search points

Reference: "The FAIR Guiding Principles for scientific data management and stewardship" Wilkinson, M. D. et al. The FAIR Guiding Principles for scientific data management and stewardship. Sci Data 3, 160018 (2016). See also Module 1.

Findable, Accessible, Interoperable and Reusable (FAIR)

How to Search for Open Code

A successful search for open code demands a clearly defined purpose. Developers must first determine the tasks they expect their code to carry out. The requirements associated with these tasks can determine the best suited programming language.



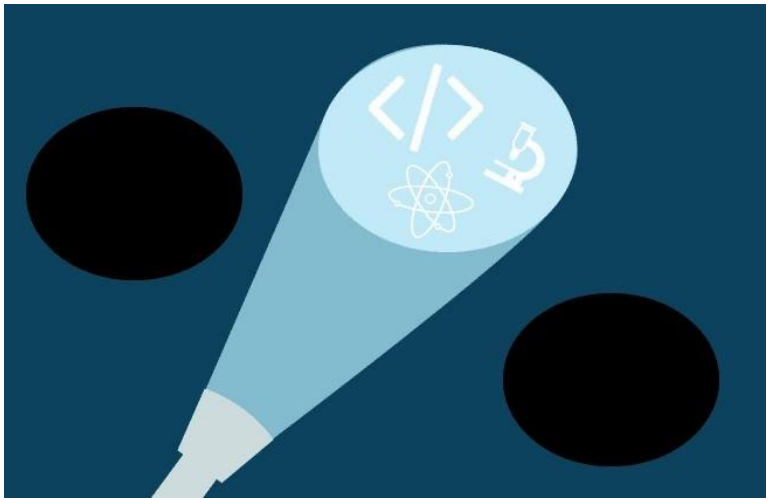
Know Where to Search

The open software ecosystem is vast, organic, multifaceted, and highly distributed.

If you are looking for scientific software, community standards increasingly require code to be published and linked to scientific papers.

Thus, the scientific literature and its ancillary code archives are increasingly a great place to look for scientific open code.

Most open code is not developed by or for scientists. However, open code enables research every day.



There are several popular search engines for code snippets. First, you can simply search on Google. Other commonly used search engines include GitHub Code Search and Stack Overflow. These search engines allow you to search for specific code snippets by programming language, keyword, or other criteria. GitHub Code Search allows you to search GitHub, a popular code repository for scientific software. Stack Overflow allows you to search forums, where users discuss solutions to coding problems.

Examples on how and where to search

- Github
- Google labs (Colabs)
- Anaconda
- Python
- Caggle.com
- Python Library sites (matplotlib, seaborn, etc.)

Four General Considerations for Assessing Open Software

Software assessment criteria are similar, for any level of openness:

- Functionality: Will it be useful for your scientific problem?
- Interoperability: How hard will it be to use?
- Security: Is it safe? Would using the software create a security risk?
- Licenses/restrictions: Can you use it? Is it legal to use the software in your project?

Functionality: Assessing Scientific Utility

- Does the software meet your scientific needs?**
- Does it address your specific science question?
- Do studies similar to yours use it?
- What papers cite it and how do they use it?
- Talk to your advisors or colleagues that might have experience with it.

Testing the scientific compatibility

- Does the software contain scientific test cases? If so, reproduce a case that is applicable to your problem; make sure the results are as expected.
- If you've done similar scientific analysis/modeling previously, reproduce your prior results with the new software. Are the results consistent?
- Incrementally modify a given test case to address new scientific questions. Alternatively, develop your own case, if necessary, following relevant examples.

Interoperability: Ease of Use

Is the code written in a language that you are familiar with?
It can be easier to use coding languages that you are familiar with, then import the code into existing software rather than try to use a new language. On the other hand, the use of existing packages and executables can accelerate your work.

Check for good documentation
Read the README file. Does the software meet your functional requirements? Are the environmental dependencies well-defined and reasonable?

Check the evidence of interoperability with other projects and codes
It is a good sign if you can find evidence that the code has been used successfully by other users that have similar scientific or technical needs.

Security: Considerations When Using Open Code

Open software is perceived to have more security risks. This is generally less of a problem for open source code than executables because the code can be audited for security vulnerabilities by the community. How can you assess security in this case?

- Consult with your institutional open software policies and IT staff
- Use authoritative reputable sources to minimize security risks
- Set strict security rules and standards when using a dependency
- Use security tools to check for vulnerabilities (e.g., Open Worldwide Application Security Project®)
- Avoid unsupported open-source software. Switch to actively developed components or develop it yourself
- Check with your latest institutional policies on using Machine Learning and Artificial Intelligence tools
- Use caution when using external tools with secure or closed access data. It may be possible for the external tool to publicly share what should be restricted information

Licenses

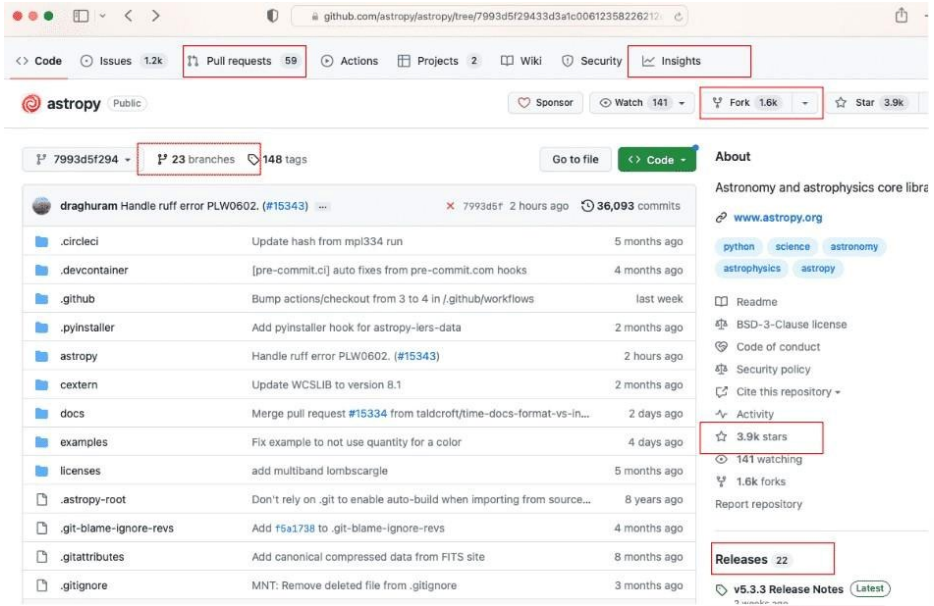
Although licensing is a nuanced subject that you will learn more about in Lesson 3, it is useful to be aware that there are generally two classes of license: permissive and non-permissive. Permissive licenses, most commonly Apache 2.0, MIT, or BSD, will generally allow you to use the code for your scientific research with little restriction, whereas non-permissive licenses such as copy-left licenses, impose substantial restrictions on how you use the code and require more careful consideration.

Source: https://elements.envato.com/licensing-2KE4GM6

Factors for Assessing the Quality of Open Source Software

GitHub

- Permits a quick scan of development activity as evidenced by the number of times the code has been downloaded or ‘forked’.
- Provides the amount of activity in a community.
- Provides insights into the quality of the software.



Source: https://github.com/astropy/astropy/pulls

Reusing Open Code

Software can be reused in a variety of ways. A software package can be executed on its own to provide a complete analysis or models depending on the input parameters. Alternatively, the package could be imported as part of a larger library to provide specific functionality. Also, code snippets can be copied into existing code, if permitted, or the code could be re-written and incorporated into new software.

If you simply intend to reuse a code snippet, continuously test that your selected code works as you expect. If you are reusing a more complex code, there are additional considerations.

Selecting the Appropriate Version for Reuse

Consider the following when selecting among multiple versions of open source software.

Use the latest stable release when possible	Developers often release developmental versions that include new features or bug fixes that are not fully tested.
Determine the origin of the version you intend to use	Determine whether the version you intend to use comes from a modified open-source project or from its original source project. With this information, determine which source is more appropriate for your project.
Check for issues and bugs	Find current information on issues or bugs by checking release notes, issue trackers, and developer forums.

Resolve Problems in Reusing Software

- Implement tests to verify that the software performs as expected in your application.
- If you run into problems, revisit the release notes, issue tracker, and/or user/developer forums.
- Don't be afraid to ask experienced colleagues for help.
- It is better to seek and obtain help in a public forum than in private (eg. email). Part of open science is working in the open. Often you may find through a search that other users have similar questions. Someone may have already offered a solution. If not, it is likely that others will benefit from your question being answered in public.

Activity 2.1

Discuss specific examples of ways to get help using open software. What has worked or not worked.

Overview of APA Guidelines

Citing work in APA format is crucial because it:

1. Acknowledges Sources: Giving proper credit to original authors and avoiding plagiarism.
2. Enables Verification: Allowing readers to locate the original sources for further reading or verification.
3. Demonstrates Credibility: Showing that the research is grounded in existing literature and contributing to scholarly dialogue.

By adhering to APA format, researchers uphold academic integrity and contribute to a coherent and credible scholarly environment.

Considerations in Citing Open Code

Question to consider when deciding whether which code to cite:

- Did the code play a critical part in your research?
- Did the code provide something novel?
- Do the software licensing terms and conditions require it?

Ideally, use and cite code that is archived in a long-term repository with a persistent DOI, which provides a persistent identifier/link for research outputs.

Follow the guidance about the preferred citation format, which may appear in a README or a CITATION file.

URLs (e.g., Stack Overflow) and active repositories (e.g., on GitHub) are mutable but can be used if there is no alternative. In most cases, a code snippet on Stack Overflow does not constitute a citable research contribution. However, an author can still decide to cite it if they chose.

See the journal where you are publishing if they have any specific instructions on how to cite software (e.g., AAS Software Citation Suggestions).

Engage with the Community

Tell other users and the developers about your experiences.

- Don't be afraid to ask the community and the developers any questions about the software you need answered
    - It's a sign of good support if the developers are friendly and approachable
    - Asking a developer can indicate the type of responses you can expect from them in the future, as well as fill in any knowledge gaps
- If you modify or improve open source software, consider contributing those improvements to the main branch so other users can benefit.

Lesson 2: Summary

- Open code exists in a vast, organic, and distributed ecosystem. Discovering Open Code depends on defining your requirements, knowing where to look, and developers using FAIR principles.
- Scientific papers are now a good place to discover scientific Open Code, since many journals require the code used in the paper to be linked via a DOI.
- Before use, it is important to assess open software for functionality, quality, interoperability, security, and license/reuse restrictions. Your first step should be to look for a README file.
- When reusing open software, use the latest supported version and test the software to ensure it functions as expected. If problems arise, reach out to the developers or user community, ideally via a public forum.
- It is important to cite and acknowledge open software that significantly contributes to your work, as well as share your lessons learned and any contributions with the developers and user community.

Lesson 2: Knowledge Check (1 of 3)

Discovering open software successfully depends on which of the following:

Select all that apply.

A. Well defined requirements
B. Knowing where to search
C. FAIR open software exists to meet your needs
D. All of the above

Lesson 2: Knowledge Check (1 of 3)

Discovering open software successfully depends on which of the following:

Select all that apply.

A. Well defined requirements
B. Knowing where to search
C. FAIR open software exists to meet your needs
D. **All of the above**

Lesson 2: Knowledge Check (2 of 3)

Read the statement and decide whether it's true or false:

It is best to reach out to the developers of open access software via private communication if you run into problems.

- True
- False

Lesson 2: Knowledge Check (2 of 3)

Read the statement and decide whether it's true or false:

It is best to reach out to the developers of open access software via private communication if you run into problems.

- True
- **False**

Lesson 2: Knowledge Check (3 of 3)

When citing Open Code, it is best practice to cite:

A. The primary working repository, e.g. on GitHub. It has the most recent version of the code, including any updates since your paper was written.

A. A long-term code repository linked to a DOI, e.g. on Zenodo. This repository contains a persistent version of the code that you used.

Lesson 2: Knowledge Check (3 of 3)

When citing Open Code, it is best practice to cite:

A. The primary working repository, e.g. on GitHub. It has the most recent version of the code, including any updates since your paper was written.

A. A long-term code repository linked to a DOI, e.g. on Zenodo. This repository contains a persistent version of the code that you used.

10 Minutes Break

Module 4 - Lesson 3

Making Open Code

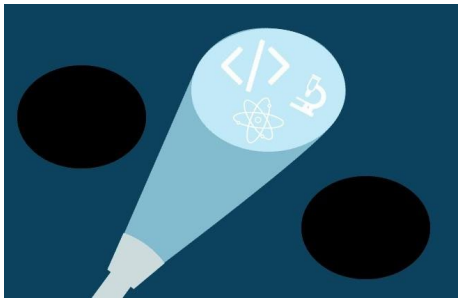
Lesson 3: Learning Objectives

- Describe the key considerations when planning a new open software project.
- List three reasons for projects to use version control.
- Explain the purpose and recall general information typically included in a README file.
- List the differences between permissive and protective open-source software licenses.
- Be able to select a license for your code
- Explain best practices in software development that support transparency, inclusion, and reproducibility.

How do We Plan for Making Code?

When starting a project, it is useful to answer the following questions:

1. What problem are you trying to solve, and are others in my community facing it as well?
2. Are there existing solutions? (In Lesson 2, we explored how to look for existing solutions.)
3. Did you find code that was close to what you want but didn't quite meet your needs?



You could potentially contribute to existing code instead of writing something new.

Even if a solution already exists, there might be good reasons to develop your own code. Instances include:

- The code is written in a different programming language than you are familiar with.
- The license is not open enough to adopt it.
- To try new techniques or to develop a deeper understanding of the problem.

You'll have to decide what's the best approach for you!

Starting a New Project

When starting a new project, the key things to consider are:

1. Define the project scope, its primary features and any limitations, and the intended audience.
2. Consider resources required for the software to run. Will it be on a personal computer, a high-performance computing server, or on the cloud?
3. How will it be managed?

Organizing a Project

Make it public! Regardless how small or big of a project, public projects have many advantages:

- Enables collaborations, invites constructive feedback, teaches others, etc. Pick a platform with version control (e.g. GitHub, BitBucket)

Pick a name!

- Make sure there are no duplicates, avoid trademarks or embarrassing names you'll regret

Choose a programming language. Languages have strengths and weaknesses; which are most important for your project?

- Think about which languages you are most experienced with, the limitations of your computing environment, your potential collaborators

Document your code as you go:

- You are you own best collaborator, and documenting will help the future you as much as others.

Explain to others what they can do with your code

- There are many ways to communicate your expectations: licenses, code of conduct, contributors, etc.

Source: https://xkcd.com/1459/

Importance of Version Control

Your code will evolve over time, no doubt. It's good to keep track of these changes either locally or online.

Version control enables the following:

- Helps developers keep track of changes to a project's code (as well as supplemental files and documentation) over the entire course of a project's evolution.
- Revisions to a project's files can be tracked, including contributions made by different people.
- Undesirable changes (like errors or bugs) can be reverted at any time.

Common version control software include git (https://git-scm.com) and svn (https://subversion.apache.org/)

Version control software is used with version control platforms online for public sharing. For example, git is used in GitHub (https://github.com/) and BitBucket (https://bitbucket.org/).

Describing Our Code to Others

The following information is helpful to add to the Project README especially if they are not listed elsewhere:

- A list of any code dependencies the software has, e.g. "Numpy, kitten-rng, and human- readable must be installed to run this software."
- How to install and a brief description of how to run the software.
- Detailed description of the software, especially if there is no external documentation.
- Examples of how to use the software.
- Acknowledgement of team members or sources of support.

Contributor Guidelines

The *CONTRIBUTING* file gives information about how to contribute to the project. It details how the contribution process works and what type of contributions are needed. While not every project has a *CONTRIBUTING* file, the existence of one is a clear indicator that contributions are welcomed.

Code of Conduct

The code of conduct sets ground rules for participants' behavior and helps to facilitate a friendly, welcoming environment. While not every project has a *CODE_OF_CONDUCT* file, its presence signals that this is a welcoming project to contribute to.

Code Documentation

Code Level Documentation for the Developer

Your software should be documented within the source code. Each function should have comments at the start that briefly state, in plain language, what the function is for. This is not only for other developers, but yourself a week later when you forgot what you wrote.

Code Level Documentation for the User

If you are developing code that you expect others to use, produce a manual on how to use the code. As code constantly develops, it is much easier to document while or even before you write any code.
Example code documentation tools include (https://www.sphinx-doc.org/) or MarkDown (https://www.markdownguide.org/)

Programming and Documenting

Establishing a Development Environment - Establishing an appropriate development environment will help you write good, clean code and will help you maintain the project as it evolves.

- Configure any necessary tools for writing the code. Perhaps an IDE (Integrated Development Environment) or text editor. Some popular examples include VS Code, Pycharm, R Studio, Xcode.
- Set up a package manager. For example, for Python, one could use 'anaconda' or 'poetry'.
- Create a virtual environment specific to your project to isolate its dependencies (and their versions) from those used for other projects

Structuring Files and Folders - How you structure the files in your project from the beginning will contribute to the success of the final results.

What License Should We Choose for Our Code?

Licensing Considerations when Using Open Software

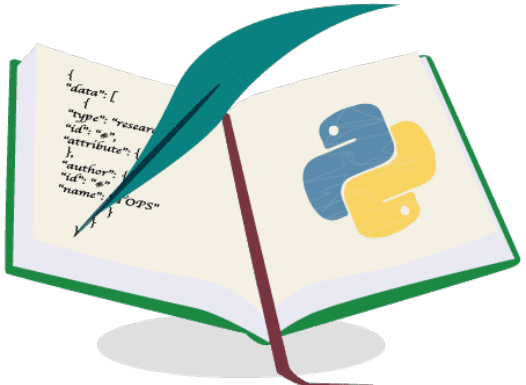
A software license is a legal document that states the rights of the developer and user of a piece of software.

An open source license is a type of software license, approved by the Open Source Initiative (OSI) as compliant with the Open Source Definition. An open source license grants permissions for anyone to inspect, use, modify, and distribute the software's source code for any purpose.

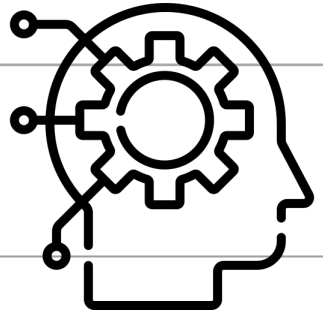
Licenses ensure that developers receive credit and control over how their work is used. Without a license, software is assumed copyrighted and without permissions. Programmers include licenses to allow reuse.

Licenses take various forms in order to outline:

- Contractual obligations (if any exist) between the developer and user.
- What the user may do with the software.
- To whom the user may distribute the software (if any such right exists).
- Length of time the user has the right to use the software.

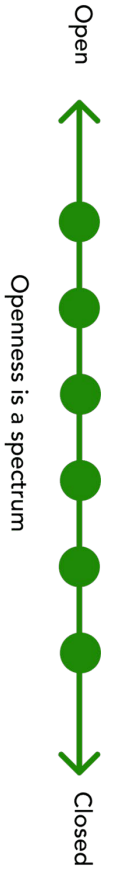


Some Common Types of Software License



Openness is a spectrum

Public Domain	Anyone free to use.
Lesser General Domain	Can link to open source libraries, and code can be licensed under any license type.
Permissive	Gives users wide but not complete latitude to reuse/relicense.
Non-permissive	Allows users to reuse, but also gives users the responsibility to share their changes with the community.
Copyleft	Can be distributed or modified if all the code involved is licensed under the same license.
Proprietary	Cannot be copied, modified, or distributed.



Activity: True or False

A software license states the rights of the developer and user for a piece of software.

- True
- False

Without a license, software is assumed copyrighted and without permissions.

- True
- False

Anyone is free to use software with a "permissive" license without restriction.

- True
- False

Users are not allowed to copy and modify any software with a copyleft license.

- True
- False

Activity 3.1

1) Chicken or egg: start coding, or start planning for code? Which do you prefer comes first? *(No good or bad answer)*
2) What's your own preference for license? Have you talked with your collaborators, your manager, your institution about this?
3) Who owns your code right now?
4) Which coder are you: retain full control of software edits, or community of equals? *(No good or bad answer)*

Programming Best Practices

- Code Review
- Testing
- Minimizing the Risk of Security Vulnerabilities
- Creating FAIR Software

Findable, Accessible, Interoperable and Reusable (FAIR)

Code Review

Code benefits from peer review in the same way as science. Having someone else read over your code and test it is one of the best ways to improve the quality of the code.

Many version control platforms have built in tools that enable developers to review, comment, and iterate on each other's code. These can be done in the open and allow anyone to comment.

Minimizing the Risk of Security Vulnerabilities

- Ensure minimal, DRY (Don’t repeat yourself) code (easier to maintain and fix).
- Use global variables or key managers for credentials. Never include credentials in your code.
- Use well-tested and maintained dependencies. In packages that you maintain, keep the list of dependencies up to date.
- Create software with tools that provide automated scanning and auditing.
- If there are unsupported dependencies that you rely on, assess them to determine how they might introduce security risks and whether it would be appropriate to switch to a different package.

Creating FAIR Software

FINDABLE	ACCESSIBLE	INTER-OPERABLE	REUSABLE
Software includes a persistent and unique identifier and rich metadata, so it is easy for humans and machines to find.	Software is retrievable from its identifier via standard communication protocols.	Software interoperates with other software; it exchanges data and/or metadata via community standards.	Fully described metadata with provenance, meeting community standards. License permits reuse.

Lesson 3: Knowledge Check (1 of 5)

Which of the following should be considered when planning an open software project? Select all that apply.

A. The intended user audience.
B. What protocol will be used to sync changes between individual contributors and the central repository.
C. The programming language to be used.
D. Who will financially benefit from sales of the software.

Lesson 3: Knowledge Check (1 of 5)

Which of the following should be considered when planning an open software project? Select all that apply.

A. The intended user audience.
B. What protocol will be used to sync changes between individual contributors and the central repository.
C. The programming language to be used.
D. Who will financially benefit from sales of the software.

Lesson 3: Knowledge Check (2 of 5)

Which of the following is a benefit of using a version control system in your software?

A. New changes are automatically tracked.
B. Different contributors can add or edit code at the same time.
C. Undesirable changes can be quickly reverted.
D. All of the above.

Lesson 3: Knowledge Check (2 of 5)

Which of the following is a benefit of using a version control system in your software?

A. New changes are automatically tracked.
B. Different contributors can add or edit code at the same time.
C. Undesirable changes can be quickly reverted.
D. All of the above.

Lesson 3: Knowledge Check (3 of 5)

Select two items that are good to include in a README file from the list below:

A. Installation/compilation instructions
B. Code development history
C. The most important portions of the code
D. Usage instructions and example output

Lesson 3: Knowledge Check (3 of 5)

Select two items that are good to include in a README file from the list below:

A. Installation/compilation instructions
B. Code development history
C. The most important portions of the code
D. Usage instructions and example output

Lesson 3: Knowledge Check (4 of 5)

Which of the following licenses allows users to reuse, but also require users to share their changes with the community using the same license?

A. Public Domain
B. Lesser general domain
C. Permissive
D. Protective License
E. Commercial

Lesson 3: Knowledge Check (4 of 5)

Which of the following licenses allows users to reuse, but also require users to share their changes with the community using the same license?

A. Public Domain
B. Lesser general domain
C. Permissive
D. Protective License
E. Commercial

Lesson 3: Knowledge Check (5 of 5)

Which of the following practices makes your project more inclusive?

A. Including a Code of Conduct.
B. Referencing historical events in the name of your project.
C. Following standards for the programming language being used.
D. Developing the project privately.
E. Including a Guideline for Contributors.

Lesson 3: Knowledge Check (5 of 5)

Which of the following practices makes your project more inclusive?

A. Including a Code of Conduct.
B. Referencing historical events in the name of your project.
C. Following standards for the programming language being used.
D. Developing the project privately.
E. Including a Guideline for Contributors.

Module 4 - Lesson 4

Sharing Open Code

Lesson 4: Learning Objectives

After completing this lesson, you should be able to:

- Describe what it means to share code: for archiving or for code development.
- Evaluate whether you should share your code and list important security considerations.
- Describe best practices for when and where to share code.
- Recall commonly used practices to help others reuse your code.
- List the roles and responsibilities for sharing and maintaining shared code.

Planning to Share Your Code

What Does it Mean to "Share" Your Code?

Open Source Code Development

Public, Dynamic, collaborative, e.g. on Github, Gitlab, etc.

Long-term Archiving

Public, Static, long-term, for reproducibility, citable, e.g. with DOI on Zenodo

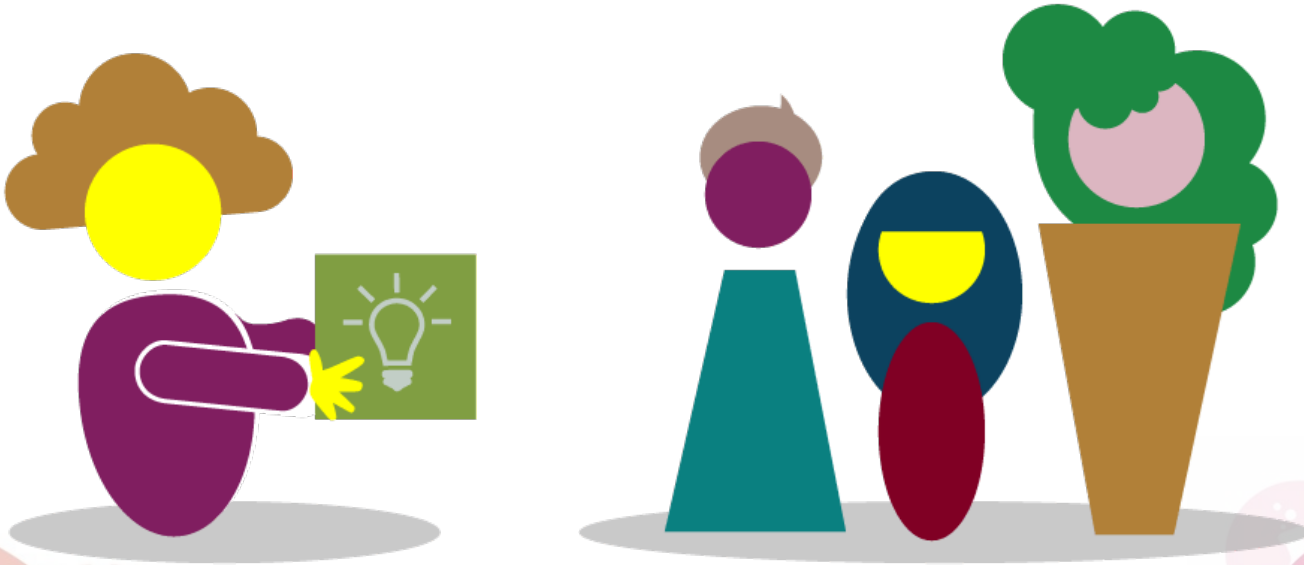
Should You Share?

Consider legal and security concerns, e.g. intellectual property/licensing

Write a Software Management Plan (SMP)

Consider the **what**, **when**, **where**, **how**, and **who** of sharing your code

Should you share?



Legal and Security Concerns (1 of 3)

Is it legal to share your code?

Questions to think about:

- Who owns the software? Are you incorporating the work of others?
- Does sharing (or not) violate institutional or funding agency policies?
- Do laws or regulations govern the sharing of intellectual property?
- What license is required?

Be aware your organization and funding agencies' policies!

Legal and Security Concerns (2 of 3)

Are you funded by a US government grant?

Agencies may require you to share your code,
even code just used for visualizing data in a journal article.

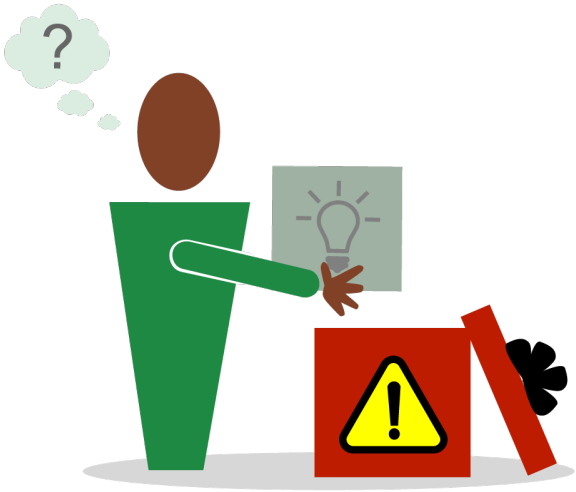
Legal and Security Concerns (3 of 3)

Sharing code on the internet raises security concerns. Could code somehow be manipulated by bad actors to exploit security vulnerabilities leading to costly damages?

Questions to consider:

Is the repository you want to contribute to reputable?

Are there any security-related issues with the code?



Be aware of your organization's IT security policies!

Practicalities of sharing code:

When, Where, How and Who?

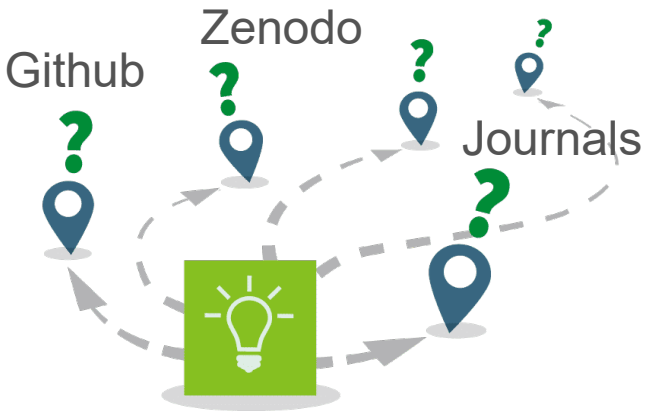
When: The Schedule for Code Sharing and Archiving

Are you writing code for a scientific paper?

Some funding agencies and journals require that the code needed to reproduce or validate the results is publicly archived **at the same time as or before the paper**.

When to share depends on the project and institutional and funding agency policies, all of which vary. Develop plans that account for these policies ahead of time.

Where: Where To Share Open Code



Do I need to put my code on Github?

Journals and funding agencies may require long-term archiving and a persistent identifier, e.g.

- Directly with the journal
- On Zenodo with a DOI

Sharing on Github is encouraged, but
Github repos are not archives like Zenodo.
They do not automatically provide a DOI.

How: How to Enable Reuse of Code

Assign a license

Make it citable

Step-by-step documentation is available in the online version of this course and on Github.

Consider adding contributor guidelines, e.g. in CONTRIBUTING and CODE_OF_CONDUCT files.

Github examples

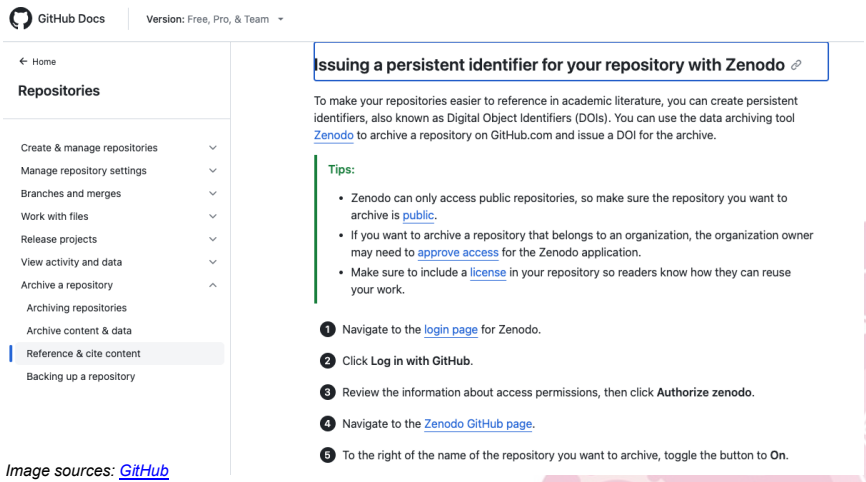
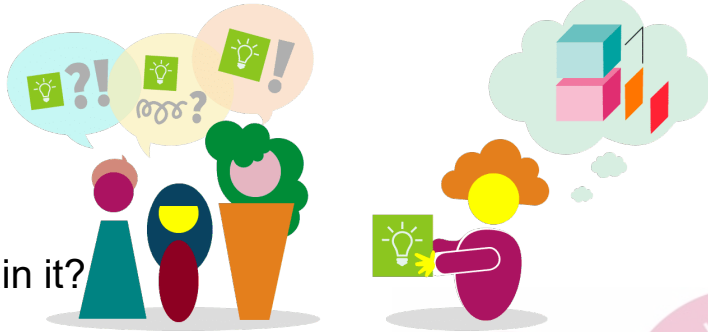


Image sources: GitHub

Who: Roles and Responsibilities of the Team Members in Implementing the SMP

Software Management Plan

Whether you're initially working on a new project by yourself or you already have a team, it is important to consider the tasks involved in sharing the code and who will do them.



- Who will add the code to a public repository?
- Who will take care of code documentation?
- Who will help with code reuse?
- Will the software be maintained? Who will maintain it?

If the software will be maintained (or even if not), consider how or who to handoff the software maintenance if need be.

Lesson 4: Knowledge Check (1 of 6)

Read the statement and decide whether it's true or false:

I don’t need to share my code if I don't plan to continue developing it.

- True
- False

Lesson 4: Knowledge Check (1 of 6)

Read the statement and decide whether it's true or false:

I don’t need to share my code if I don't plan to continue developing it.

- True
- False

Lesson 4: Knowledge Check (2 of 6)

Read the statement and decide whether it's true or false:

Adding code to a GitHub repository is sufficient for archiving my code.

- True
- False

Lesson 4: Knowledge Check (2 of 6)

Read the statement and decide whether it's true or false:

Adding code to a GitHub repository is sufficient for archiving my code.

- True
- False

Lesson 4: Knowledge Check (3 of 6)

Read the statement and decide whether it's true or false:

Organization and government software-sharing policies follow a standard practice.

- True
- False

Lesson 4: Knowledge Check (3 of 6)

Read the statement and decide whether it's true or false:

Organization and government software-sharing policies follow a standard practice.

- True
- False

Lesson 4: Knowledge Check (4 of 6)

Read the statement and decide whether it's true or false:

Publishing your software to a software repository used by common package managers makes it easier for users to install your software.

- True
- False

Lesson 4: Knowledge Check (4 of 6)

Read the statement and decide whether it's true or false:

Publishing your software to a software repository used by common package managers makes it easier for users to install your software.

- True
- False

Lesson 4: Knowledge Check (5 of 6)

Which, if any, of the following are ways you can help others to reuse your code? Select all that apply.

A. Assign an appropriate license
B. Add a file named "CONTRIBUTING" with contributor guidelines
C. Add a "CITATION" file with citation information

Lesson 4: Knowledge Check (5 of 6)

Which, if any, of the following are ways you can help others to reuse your code? Select all that apply.

A. Assign an appropriate license
B. Add a file named "CONTRIBUTING" with contributor guidelines
C. Add a "CITATION" file with citation information

Lesson 4: Knowledge Check (6 of 6)

Which of the following are roles that you should plan for when writing a SMP? Select all that apply.

A. Who will help maintain the software
B. Who will create the repository and add the necessary files
C. Who will contribute to the software after it is shared
D. Who will add documentation to the software

Lesson 4: Knowledge Check (6 of 6)

Which of the following are roles that you should plan for when writing a SMP? Select all that apply.

A. Who will help maintain the software
B. Who will create the repository and add the necessary files
C. Who will contribute to the software after it is shared
D. Who will add documentation to the software

10 Minutes Break

Module 4 - Lesson 5

From Theory to Practice

Lesson 5: Learning Objectives

After completing this lesson, you should be able to:

- Recall the definition of a software management plan, potentially as part of an open science and data management plan, and where to find helpful resources.
- List ways to engage with and contribute to open software communities.

Open Science and Data Management Plans

> A NASA Open Science and Data Management Plan (OSDMP) describes how the scientific information that will be produced from NASA-funded scientific activities will be managed and made openly available. The OSDMP should include sections on data management, software management, and publication sharing.
>
> **https://science.nasa.gov/researchers/sara/faqs/**

How Do We Plan for Making our Code Open?

SMP is required	SMP is not required
You need a SMP to: • Propose for funding (e.g., NASA, NSF, and likely everywhere soon) • Collaborate on a team that intends to release code to the public • Successfully manage any large mission or project	You probably don't need a SMP if you are working on: • A paper by yourself (or with a very small group of collaborators) • The initial exploration of ideas or experimentation with analysis code • Education-focused activities

Engage and Build Communities

PANGEO

NUMFOCUS

OPEN CODE = BETTER SCIENCE

PyData

ROpenSci

R Ladies

pyladies

RSE

pyOpenSci

Contribute to Open-Source Software

Add New Features	The most obvious case for contributing to open software is enhancing its usability by adding new features.
Fix Bugs	Alternatively, you can reply to an already opened issue by fixing it.
Report Issues and Make Suggestions About Improving Code	Reporting an issue is a valuable contribution even if you don't know how to fix it. For example, you might be using a different browser in which the software has not been tested yet, have discovered a particularly uninformative error message, be colorblind or be otherwise able to feed a valuable user experience back to the developers that can help to improve the overall usability of the software.

Contribute to Open-Source Software

Improving and Contributing to Documentation	Contributing to documentation constitutes a great starting point to contributing to open source software and is often overlooked in its importance. Writing documentations allows you to familiarize yourself with the use of the software, while helping to teach others.
Create Tutorials, Use Cases, or Visuals	Another way to contribute is to make your experience and use of the software publicly available. For example, you could create a tutorial based on your use of the software, summarize a use case or provide a summary of your use in a graphic. This part of contribution is particularly appealing as it does not create much extra work to just publish what you have used the software for.
Improve Layout, Automatization, Structure of Code	Apart from creating new code, a good way to contribute to open source software can also be to improve, restructure or automatize existing code. This is called refactoring and helps to make the software project more effective and stable.
Organize and Attend a Community Meet-Up	Another way to contribute to open source software is via community building. Many software products and toolboxes have a lively community of users that meet on a regular basis in person and online to discuss and improve the software and its use. Participating or even organizing such a meetup can be a good way to improve your knowledge of the software, get to know its community, and contribute to open source projects.
Code Review	Requests to integrate new contributions into the main code base usually require a review of the contribution by at least one other user. Similar to peer review, code review entails writing a short summary about the quality of the code and making suggestions about improvements.

Lesson 5: Summary

In this lesson, you learned:

- When a SMP should be written and that your funding organization or institution may have rules around how you develop and share your code.
- That joining software communities can be a great way to exchange knowledge and learn new skills around open code.
- That there are many ways to contribute to open code, and that not all of them require writing code.

Lesson 5: Knowledge Check (1 of 2)

Read the statement and decide whether it's true or false:

Community engagement with open software is non-hierarchical; all members of the community should be treated the same.

- True
- False

Lesson 5: Knowledge Check (1 of 2)

Read the statement and decide whether it's true or false:

Community engagement with open software is non-hierarchical; all members of the community should be treated the same.

- True
- False

Lesson 5: Knowledge Check (2 of 2)

Select the beneficial way(s) to contribute to open sources software.

A. Add new features
B. Fix bugs
C. Document your work
D. Refactoring
E. All of the above

Lesson 5: Knowledge Check (2 of 2)

Select the beneficial way(s) to contribute to open sources software.

A. Add new features
B. Fix bugs
C. Document your work
D. Refactoring
E. All of the above

Open Code Summary

Congratulations! Today we overviewed the following:

- Explained what open source software means, including the software development cycle, the benefits of open software, and some common limitations and how they are addressed.
- Discovered open source software and assess it for reuse by evaluating provided documentation, including README files and licensing details; cite the software when appropriate.
- Created an open source software management plan that includes the strategy for selecting open software dependencies and open repositories such as GIT, and how open elements including metadata, README files and version control, will be included to make the software reusable and findable.
- Evaluated whether your open source software can be shared and the best options for sharing to increase visibility.
- Listed the responsibilities a software developer has once the open source software is shared including managing legal requirements and ensuring the software is maintained.